॥श्री गणेशाय नमः॥

Data Warehousing → centralized repo of data collected from heterogeneous sources.

DW → Subject-oriented, Integrated, Time-variant, Non-volatile

ETL
Extract: from various sources (involves validation)
Transform: raw data → format the DW uses
Load: loads data into end target.

ETL based: uses a staging area beforehand
ELT based: staging area inside warehouse

Data mart: subset of data mart; focuses on some specific subject area.
Built on dimensional model;

Warehouse architecture → Top-down: Warehouse ⇄ mart, mart, mart → mining
→ Bottom-up: mart, mart, mart → warehouse → mining

{ Extraction layer (ETL) → OLAP server → Front-end layers (querying, analysis, etc.) } 3-tier architecture

OLTP: executing # of transac^ns concurrently. (on DB)
OLAP: multidim. analysis at high speeds on large vol^ms of data. (on DW)

Data cube: multi-dim array of values.
Dimension: Perspectives to analyze the subject for.
measurements (Facts): Numeric values/ aggregations to be analyzed.

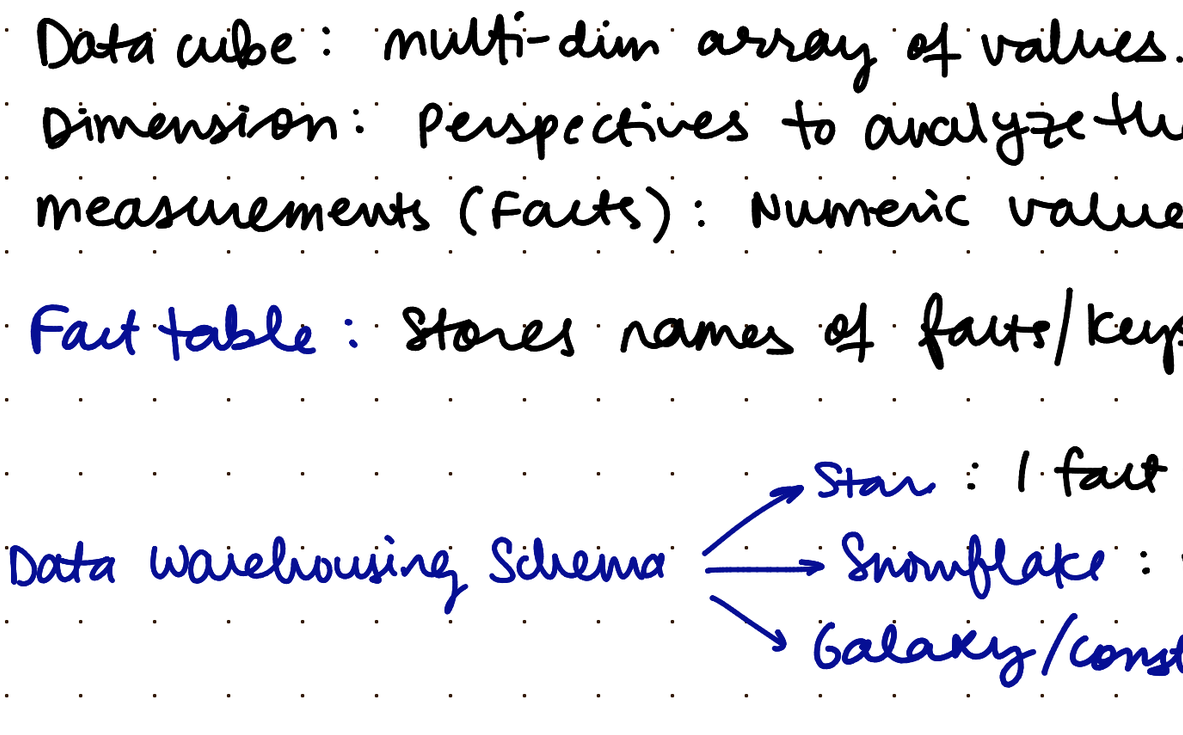
Fact table: Stores names of facts/keys referencing to dim table.

Data Warehousing Schema → Star: 1 fact table, 1 dim table per dim. (Denormalized)
→ Snowflake: multiple dim tables per dim.
→ Galaxy/constellation: multiple fact tables, shared dim tables.

concept hierarchy: mapping low-level concepts to high-level ones.

Schema-based

E.g. location: street < city < state < country



Set-group

Measures

Distributive: combining results of aggregation on subsets of data.
E.g. Summation, MAX, MIN

Algebraic: algebraic combination of other measures.
e.g. $AVG() = \frac{SUM()}{COUNT()}$

Holistic: No constant bound on storage size to describe the sub-aggregate.
e.g. mode, median.

OLAP operations

1. **Roll-up / Drill-up**: Aggregation of data cube. move up concept hierarchy.
2. **Drill-down**: less detailed → more detailed. (move down / add dim)
3. **Slice**: Performing selection on 1-dim of given cube. Data cube → sub cube.
4. **Dice**: Selection on 2/more dim.
5. **Pivot**: Rotate data axes to provide alt. representaⁿ.
6. **Drill-across**: compare measures from diff. factual tables that share same dimensions.
7. **Drill-through**: Drill through bottom-level down to its backend relational tables using SQL.

- Data
  - categorical / qualitative
    - nominal {"black", "blue"}
    - ordinal {"poor", "good", "excellent"}
  - numerical / quantitative
    - value-based
      - discrete
      - continuous
    - zero-point based
      - interval-scaled
      - ratio-scaled (true zero)

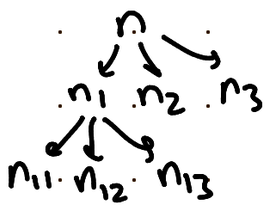
Similarity measures

- Numerical Data
  - Euclidean Distance $d(p,q) = \sqrt{\sum (p_i - q_i)^2}$
  - Manhattan distance $d(p,q) = \sum |p_i - q_i|$
  - Minkowski Distance $d(p,q) = \left(\sum |p_i - q_i|^k\right)^{1/k}$

- Data Reduction
  - Dim. reduction
  - Data compression
  - Data aggregation
  - Data numerosity
- Data cleaning
  - missing data → imputation
  - noisy data → binning (smoothing by mean/bin boundary)
  - → outlier analysis
  - → Regression
- Data Integration
- Data Transformation
  - ↳ normalizaⁿ, discretizaⁿ, compression.

- Normalization
  - min-max $x' = \frac{x - \min(x)}{\max(x) - \min(x)} \cdot (b-a) + a$
  - z-score $z = \frac{x - \mu}{\sigma}$
  - Decimal-Scaling $x_i' = \frac{x_i}{10^j}$

- Discretization
  - class information based
    - Supervised
    - Unsupervised
  - Direction-based
    - Top-down



    - Bottom-up → merging

remove potential split points by merging neighborhood values to form intervals.

Binning : top-down, unsupervised.
↳ equal width / equal freq. / both

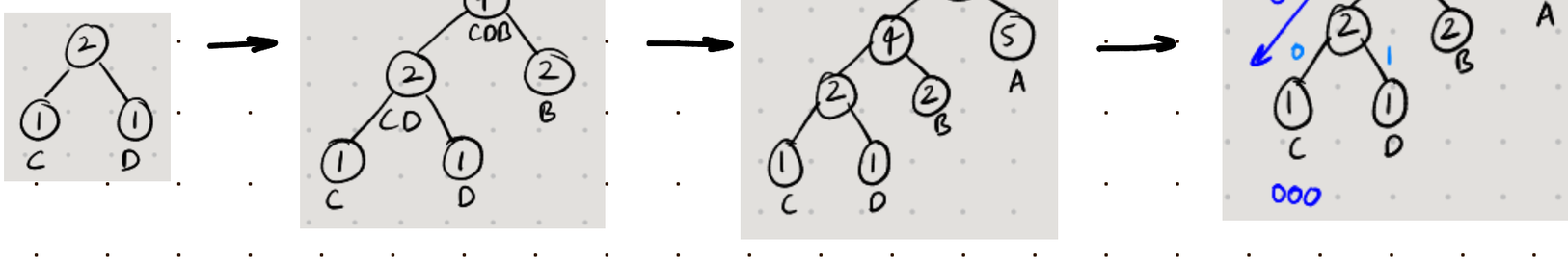
Histogram analysis : Divide data into bins and compute freq. of observans for each bin
Do recursively to get coarser (less granular) bins.

Data compression → Lossless: Huffman coding, Run-length encoding, LZW encoding
→ Lossy: Transform coding

compression ratio $\propto \frac{1}{\text{quality of reconstruction}}$

Huffman coding

(1, C), (1, D), (2, B), (5, A) } freq-tuples (sorted)



LZW encoding:

ababab ab

Initial dictionary

encoding	index	entry
	1	a
	2	b

a b ab ab ab

↑ start next traversal here
make new entry

encoding	index	entry
	1	a
	2	b
	3	ab

ab ab a bab

+ ↑ start next

encoding	index	entry
	1	a
	2	b
	3	ab
	4	ba

a ba ba ba b

+

encoding	index	entry
	1	a
	2	b
	3	ab
	4	ba
	5	aba

a b a b a b a b

+

encoding	index	entry
	1	a
	2	b
	3	ab
	4	ba
	5	aba
	6	abab

At each step, start traversal from the end of last added part, and add the next part not present in the dictionary.

encoding example:

aabb ca b cca

encoding	index	char
	1	a
	2	b
	3	c
1	4	aa
1	5	ab
2	6	bb
2	7	bc
3	8	ca
5	9	abc
3	10	cc

aa bb ca b cca ***

1 1 2 2 3 5 3 8 — for ca

we add the left out characters at the end.

decoding: add new entry as (prev entry + single char)

1 1 2 2 3 5 3 1 — abc (1 2 3)

index	char
1	a
2	b
3	c
4	aa
5	ab
6	bb
7	bc
8	ca
9	abc
10	cc

~~1~~ ~~1~~ ~~2~~ ~~2~~ ~~3~~ ~~5~~ ~~3~~ ~~8~~

a a b b c ab c ca

aa ab bb bc ca abc cc

Transform coding :

Transformation → quantization → encoding

Sampling

Simple random sampling : each point has equal chance of being selected. SRS ⟨ with replacement / without replacement

Stratified sampling : Divide D into mutually disjoint strata. Sample from each.

Cluster sampling : Divide pop^n into clusters. Take random sample of clusters.
Take all elements from all clusters → cluster sampling.
sample from each cluster → 2-stage CS.

Systematic sampling : Pick every Kth point.

